

CPT102 Data Structures: Lecture 1 : Introduction to Data Structures

Thomas Selig

XJTLU

March 1, 2026

Content of today's lecture

Today's lecture introduces the first strand of the course: **Data Structures**

- Data structures: what? why?
- Review of arrays
- Introducing stacks and queues
- Towards implementation: Java interfaces and collections

Next week will introduce the second strand: **Algorithms**

Lecture learning outcomes

By the end of the lecture, you should be able to:

- A. Define the concept of data structures, and explain their importance in computer systems.
- B. Define and perform basic operations on arrays, stacks, and queues.
- C. Describe the concept of a Java interface and explain their role in the Java collection framework.

Data structure: what? why?

Definition

A data structure is a specialised format for organising, storing, and managing (aggregate) data in a computer so that it can be used efficiently.

- Very often, our computer programs need to store or manipulate aggregate/composite data, not just single values (e.g., integers).
 - Example: students in a university.
- Much like our course, this definition has two key strands:
 - ① “... format for organising, storing [...] data” refers to the underlying data **structure**;
 - ② “managing data”, “used efficiently” refer to the **operations** that will be performed on or with the data.
- A structure which stores aggregate data is sometimes also referred to as a *collection* or *container*.
- The data stored in the collection are the *elements*.

Activity 1: common operations

- Think of a university system such as e-bridge, which stores the data relating to students in the university.
 - Let's refer to this as a “collection of students” and name it `students`.
- What operations will be most commonly performed on `students` in the system?
 - When building/updating the system?
 - When a student or member of staff wants to look up information?

You can also use your CPT111 Week 10 Lecture notes for inspiration.

- Share your examples in the Etherpad activity [here](#).

Why different data structures?

Question

Why do we need different types of data structures? Can we not just use arrays (or lists) for everything?

- In theory, yes!
 - The fundamental model of a computer's storage/memory is just an array of integers (or bits).
- However, arrays have limitations:
 - Potential memory leaks or waste;
 - Inefficient for certain operations (e.g., searching);
 - Lack the richness to express more complex, highly structured, data (e.g., graphs).

Selecting the correct data structure

A key learning outcome of this course is for you to be able to select an appropriate data structure to solve a given problem.

In general, two key considerations when selecting (or developing) a data structure:

- ① The intrinsic **properties** of our data structure.
 - Do we want possible duplicated data in our structure or not?
 - Do we want our structure to be ordered/unordered? sorted/unsorted?
 - Different structures force different constraints.
- ② The typical **operations** we need to perform, e.g. iteration, add, remove, access (get), search.
 - Different structures have different speeds (complexities – next week) for these.
 - Which operation(s) do we need most frequently?

Activity 2: data structure examples

- We usually use a data structure to represent data items that form a natural group, e.g.:
 - A child's toys (collection of toys);
 - Student cohort (collection of students).
- Think of some examples of computer programs that you use every day: internet search, recommendation algorithm (Tiktok, Red Note, etc.), online shopping app, and others.
- What collections do these manipulate/output? What are the elements in the collection? What are the properties (duplicates allowed? ordered or not?) of the collection?
- Share thoughts in Etherpad activity here.

In this course, we will learn a number of common data structures, including:

- List-based structures such as ArrayList, LinkedList, Stack, Queue;
 - Also called *linear* data structures;
- Tree-based structures such as Heap, Binary Search Tree, AVL (Adelson-Velsky and Landis) Tree;
- Graphs and Networks;
- Hashing-based structures such as HashSet, HashMap.

Throughout, focus on algorithms related to these.

Basic example: Bag

A **bag** is perhaps the simplest possible data structure.

- A bag is a simple, unordered, collection of elements, with no underlying constraints.
 - Can contain duplicates.
- Very few operations available. In the simplest model, can only:
 - Add new elements;
 - Inspect (and remove) a random element.
- In fact, very rarely used in computing, since nearly all data has more structure (+ underlying computer model).



Arrays: definition

- An array is a simple ordered (linear) collection of data.
- Arrays store their elements in **consecutive memory locations** and refer to them by their **index**.
 - Arrays are therefore **contiguous** blocks of memory in a computer.
- An array has a fixed number of elements, which is called its **size** or **length**.

Example

- This is an array of *Strings*:

It's	a	small	world	after	all
0	1	2	3	4	5

- In most programming languages (including Java, Python, C/C++, etc.), arrays are *0-indexed*. This array has a *length* of 6.
- When writing code or pseudo-code, we will generally use `array[i]` to denote the *i*-th element of an array.
- In fact, this array would store *references* to the String values, rather than the values themselves.
 - Why do you think that is?

Array operations

- Iteration follows order of indices, allows for searching, etc.
- Indexing allows for positional operations: retrieve or modify the value at a given index, search by index, extract sub-array from `startIndex` to `endIndex`, etc.
- Because arrays have a fixed length, we cannot add or remove elements from an array.

Arrays: pros and cons

- Because an array is a contiguous block of memory, we have *fast access* (by index). To access data stored at `array[i]`, we simply need to:
 - 1 Retrieve memory address of `array[0]` (typically this is the reference of the array itself);
 - 2 Increment this memory address by `i` times the (memory) size of an element of the array.

Essentially, “skip” to the `i`-th element directly. FAST! (demo)

- For searching, we need to iterate over the whole array. SLOW!
 - If this is what you need to do most often, there are better choices (e.g., hash sets).
- If we want to add or remove elements, we have to make a new array, and copy everything into it. WASTEFUL!

Stacks and queues

Stacks and queues are two array-based (or list-based) data structures.

- Stacks and queues are both used for processing elements one-by-one.
- They are **resizeable**: elements can be added and removed.
 - Once an element has been processed, it is removed from the structure.
- The way/order in which elements are added and removed is different for both.
 - Stacks are **Last In, First Out** (LIFO): the element which is processed is always the last one that was added to the stack.
 - Queues are **First In, First Out** (FIFO): the element which is processed is always the first one that was added to the queue.

Stacks: definition

Definition

A **stack** is a linear (list-based) data structure in which elements may be inserted or removed from only one end.

- Follows FILO order.
- Model is an array where one end is “open”, with additional space for new elements.
- Open end is usually referred to as the *top* of the stack.
- On the right is a stack of integers.

⋮
5
42
-12
2
10

Stacks: operations

- Stacks support three standard operations:
 - `push(e)`: adds the new element `e` to the stack.
 - `pop()`: removes (and returns) the top element of the stack.
 - `peek()`: inspects (returns, but doesn't remove) the top element of the stack.
- All three operations are FAST, except when resizing is required (too much or not enough free space at the top).
- Usually, there is also a `size()` operation that returns the number of elements in the stack.

Stacks: application examples

Stacks are used in a number of computer-based tasks. For example:

- Processing files of structured (nested) data.
 - E.g., reading files with structured markup (HTML, XML, etc.).
- Program execution, e.g. working on subtasks, then returning to previous task.
- Undo in editors.
- Expression evaluation: $(6 + 4) * ((12.1 * \exp(15)) - (\log(20) / 38))$.
- And more (see exercise sheet).

Example: checking balanced expression

- Consider an expression with different delimiters: ([{
 - For example: $a * (b + c / [f(x) * \{g(y) - h(z)\}])$
- How can we check the expression is *well-balanced*?
 - Every opening delimiter must have a matching closing delimiter.
 - Every “sub-expression” enclosed inside such a matching opening-closing pair must also be well-balanced.
- We can use a stack! How?

Checking balanced expressions using a stack

Here is a routine to check if an expression is well-balanced.

- ❶ Start with an empty stack.
- ❷ Read the expression from left-to-right:
 - If we encounter an opening delimiter, push it to the stack.
 - If we encounter a closing delimiter, pop the stack; if stack is empty, FAIL.
 - If the popped (opening) delimiter matches the current closing delimiter, we have found a matching pair, and continue.
 - Otherwise, FAIL.
- ❸ Once we have finished reading the entire expression:
 - If the stack still has elements, FAIL.
 - Otherwise, if the stack is empty, SUCCEED.

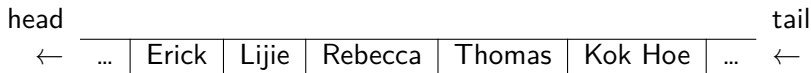
The above is an example of an **algorithm**, which is the second key strand of this course (more next week).

Queues: definition

Definition

A (standard) **queue** is a linear data structure in which elements are inserted only at one end, and removed only from the other end.

- Follows FIFO order.
- Model is an array where both ends are open.
- The *end* or *tail* of the queue is where elements are added, the *front* or *head* where elements are removed from.
- Below is a queue of people.



Queues: operations

- Essentially the same as for stacks, but different names and effects.
 - `enqueue(e)`: adds the new element `e` to the tail of the queue.
 - `dequeue()`: removes (and returns) the element at the head of the queue.
 - `peek()`: inspects (returns, but doesn't remove) the element at the head of the queue.
- All three operations are FAST, except when resizing is required (too much or not enough free space at the head or tail).
- As with stacks, there is also usually a `size()` operation that returns the number of elements in the queue.

N.B.: The names `enqueue()` and `dequeue()` are the most standard, but not universal. Java uses `offer()` and `poll()`, and some other languages also differ.

Queues: application examples

Most commonly used when some task/program/system needs to hold elements prior to processing. For example:

- Task scheduling for Operating Systems.
 - E.g.: CPU process scheduling, print spooling.
- Programs which simulate/represent queues in the real world.
 - E.g.: customer service or ticket booking systems, ride-car hailing (Didi), food delivery apps (ele.me).
- Some algorithms that we will see later in this course use queues, e.g. DFS on graphs.
- Applications that require concurrency and multi-threading.

N.B.: Not all of these follow FIFO order. There is a variant called **priority queue** where elements/tasks are stored in order of priority, and the most urgent is always processed first. More on this later in the semester.

Towards implementation

- So far most of what we've covered has been conceptual:
 - Models of data structures;
 - An algorithm for checking if an expression is well-balanced.
- Most of the course will be similar:
 - Introduce data structures concepts;
 - Algorithms shown in *pseudo-code*.
- But it's important to also understand how these operate in practice.
 - The *implementation* part of the course.

Implementations in this course will be shown either in pseudo-code or in the Java programming language.

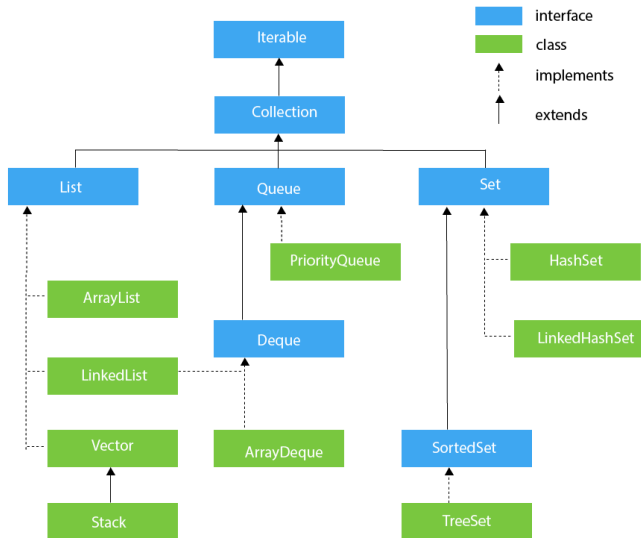
- Pseudo-code is important, since you should understand course contents independently of chosen language.
- We chose Java for continuity and familiarity: you already studied it in CPT111 last semester.
 - You should revise the basics: program structure, data types (primitives and String), control flow (if/for.while), etc.
 - We will revise some key concepts (e.g., classes) as needed.
- In this course, we will:
 - Manipulate existing structures in the **Java collection framework**;
 - Define our own custom-build Java collections.
- Today, we will:
 - Briefly introduce the collection framework;
 - Define the concept of a (Java) **interface**.

Java collection framework: what?

- From the Java tutorial here: “a unified architecture for representing and manipulating collections”.
- It contains:
 - interfaces: abstract data types that represent different sorts of containers; `List`, `Set`, `Map`, etc.;
 - implementations: concrete implementations of the interfaces; `ArrayList`, `HashSet`, `HashMap`, etc.;
 - algorithms: methods that manipulate the collections and perform useful operations on them (search, sort, add, etc.); these are *polymorphic*, i.e. same name, implementation depends on sort of collection.
- The collection framework is divided into two categories.
 - ① `Collection` objects for collections of values only.
 - ② `Map` objects for collections of pairs (`Key`, `Value`).

Today we will only look at `Collection`.

The Collection hierarchy



What do these words “interface”, “implements” mean??

Introducing Java interfaces (1)

- An interface is a blueprint, or abstract framework, for classes to use. We say that a class *implements* an interface.
- An interface provides a *contract* for implementing classes to fulfill. It does this by specifying a (usually small) number of things that any implementing class must be able to *do*.
- Interfaces can be implemented by many (very) different classes.
- The interface says only *what* things an implementing class should do, but nothing about *how* the class should do these things.

Introducing Java interfaces (2)

- *What* a class/object can *do* $\equiv$ methods in the class.
- The interface says nothing about *how* $\equiv$ no implementation details provided, just an empty method!
- We call an empty/unimplemented method an *abstract* method.
- Definition: A Java interface is a grouping of abstract methods, usually a small number.
- When a class *implements* an interface, it must *override* all of the interface's methods by providing specific implementations of them.

Interfaces in Java: syntax

```
public interface Writeable {  
  
    public static final int MAX_LINES = 1000;  
  
    public boolean write(String fileName);  
  
}
```

- Declaration is `public interface` rather than `public class`.
- The `write()` method is abstract: it has no body, only a signature followed by a terminal semi-colon.
 - Methods in an interface are always public: keyword `public` is optional.
- Since Java 8, can supply *default* implementations of interface methods (I rarely use this).
- An interface can have public, final, static variables, but never instance variables.
 - Keywords `public`, `static`, `final` are all optional.
- Often, interface names are *adjectives*, often ending in -able; a “can” relationship.

Returning to the Collection interface

- The root interface in the collection hierarchy.
- A collection is a group of objects (*elements*).
- Different implementations: with/without duplicates, ordered/unordered, etc..
- Collection methods: `add()`, `remove()`, `contains()`, `size()`, `isEmpty()`, and a few others; see here.
 - Most general methods for a collection of objects. Any collection of objects should have these methods.
 - But how they are *implemented* depends on the type of collection.
 - Example: adding to a Queue $\neq$ adding to a Stack!!
- Collection extends Iterable, which means we can *iterate* any collection.

Summary

- Concept of data structures for storing and manipulating collections of elements; Important to select appropriate structure based on its properties and the required operations.
- Review of arrays: indexing, fast access, limitations.
- Introduction to stacks and queues: conceptual models, fundamental operations (add, remove, peek, size).
- Moving towards implementation: introduction to Java interfaces and the collection framework.